

Dynamic Agentic Artifacts with CopilotKit & AG-UI

Production Architecture Guide: Layout Generation, S3 Persistence, and Resilience Patterns

Document Type: Technical Architecture Guide

Target Infrastructure: AWS (EKS / S3)

Framework Stack: CopilotKit v2, AG-UI, React

1. Executive Summary & Core Pattern

In enterprise agentic applications built on **CopilotKit v2** and the **AG-UI protocol**, agents are frequently required to produce complex analytical artifacts (e.g., performance reports, compliance portfolios, business dashboards) dynamically. Standard markdown rendering in chat interfaces fails when layouts require structural variation, such as side-by-side positioning (text on left, charts on right) or complex embedded tables.

To solve this robustly, this architecture applies the **Server-Driven UI (Component-Driven Layout) Pattern**. The agent does not generate rendering code or raw file uploads directly. Instead, it emits a strongly typed, block-based JSON schema into the synchronized **Co-Agent Shared State**. The frontend workspace consumes this state, orchestrates dynamic React components natively, handles server-side persistence via Amazon S3, and ensures high resilience across execution failure loops.

2. Dynamic Block-Based Layout Schema

To ensure total flexibility, the agent structures its analytical reasoning into sequential sections. Each section dictates its column allocation and content type. Below is the strict JSON configuration required by the orchestrator graph.

```
{
  "title": "Q3 Enterprise Analytics Portfolio",
  "sections": [
    {
      "id": "sec_01",
      "columns": [
        {
          "width": "md:col-span-1",
          "content": [
            { "type": "text", "content": "Quarterly transactional volume observed an unprecedented 42% surge following the core cluster migration in May." }
          ]
        },
        {
          "width": "md:col-span-1",
          "content": [
            { "type": "chart", "chartType": "bar", "data": [ {"label": "Apr", "value": 120}, {"label": "May", "value": 195}, {"label": "Jun", "value": 240} ] }
          ]
        }
      ]
    },
    {
      "id": "sec_02",
      "columns": [
        {
          "width": "md:col-span-2",
```

```

        "content": [
          {
            "type": "table",
            "headers": ["Service Endpoint", "SLA Target", "Actual Availability", "Status"],
            "rows": [
              ["/api/v2/checkout", "99.95%", "99.98%", "OPTIMAL"],
              ["/api/v2/auth", "99.99%", "99.91%", "DEGRADED"]
            ]
          }
        ]
      }
    ]
  }
}

```

3. Production React UI Renderer

The frontend workspace acts as a consumer of the CopilotKit state store. It maps block types to high-fidelity internal UI components while serving as the gateway to the PDF compilation and upload flow.

```

import React from 'react';
import { useCopilotState } from "@copilotkit/react-core";
import { BarVisualizer, LineVisualizer } from '@components/analytics-charts';

export function EnterpriseReportWorkspace() {
  const [state] = useCopilotState();

  if (!state.sections || state.status === 'processing') {
    return <div className="loading-spinner">Compiling artifact layout...</div>;
  }

  return (
    <div id="capture-workspace-root" className="p-8 bg-white max-w-5xl mx-auto space-y-6">
      <div className="border-b pb-4 flex justify-between items-center">
        <h1 className="text-2xl font-bold text-gray-900">{state.title}</h1>
      </div>

      {state.sections.map((section) => (
        <div key={section.id} className="grid grid-cols-1 md:grid-cols-2 gap-6">
          {section.columns.map((col, cIdx) => (
            <div key={cIdx} className="space-y-4 flex flex-col justify-center">
              {col.content.map((block, bIdx) => (
                <BlockDispatcher key={bIdx} block={block} />
              ))}
            </div>
          ))}
        </div>
      ))}
    </div>
  );
}

function BlockDispatcher({ block }) {
  switch (block.type) {

```

```

case 'text':
  return <p className="text-gray-700 text-sm leading-relaxed">{block.content}</p>;
case 'chart':
  return (
    <div className="h-64 w-full p-4 border rounded-xl">
      {block.chartType === 'bar' ? <BarVisualizer data={block.data} /> : <LineVisualizer
data={block.data} />}
    </div>
  );
case 'table':
  return (
    <div className="overflow-x-auto border rounded-lg">
      <table className="min-w-full divide-y text-xs text-left">
        <thead className="bg-gray-50 text-gray-600 uppercase font-semibold">
          <tr>{block.headers.map((h, i) => <th key={i} className="p-2 border">{h}</th>)}</tr>
tr>
          </thead>
          <tbody className="divide-y text-gray-700">
            {block.rows.map((row, rIdx) => (
              <tr key={rIdx}>{row.map((cell, cIdx) => <td key={cIdx} className="p-2
border">{cell}</td>)}</tr>
            ))}
          </tbody>
        </table>
      </div>
    );
  default: return null;
}
}

```

4. Enterprise S3 Storage Architecture & Lifecycle

Artifacts must not be retained locally or in volatile client memory. Instead, they must follow a secure, decoupled processing pipeline from client-side generation to centralized AWS S3 storage.

4.1 Secure PDF Generation and Upload Flow

1. **Client-Side PDF Capture:** The workspace DOM node (comprising SVG charts and layout columns) is captured using `html2pdf.js` or `jspdf` and converted into a binary Blob object.
2. **Pre-signed URL Acquisition:** The frontend requests a limited-lifetime IAM-restricted Amazon S3 Pre-signed URL via a backend microservice. This completely removes AWS credentials from client memory.
3. **Direct S3 Upload:** The client executes a binary `PUT` request directly to the S3 bucket via the pre-signed URL, keeping execution overhead away from the application server.
4. **State Notification:** The client passes the immutable S3 file path back to the agentic orchestrator context via CopilotKit to notify the user in chat.

Storage Tier	Retention Period	Access Pattern / IAM Scope	Optimized S3 Storage Class
In-Flight Drafts	7 Days	Read/Write via App Service Role	S3 Standard / Lifecycle to Expiration
Production Audited Reports	Indefinite	Cross-account analytical access (Read-Only)	S3 Intelligent-Tiering (Archive Enabled)

4.2 Client Execution Block (Presigned Upload Pattern)

```
import html2pdf from 'html2pdf.js';

async function handleGenerateAndUploadToS3(state, copilotContext) {
  const element = document.getElementById('capture-workspace-root');
  const opt = { margin: 0.5, filename: 'temp.pdf', image: { type: 'jpeg', quality: 0.98 },
html2canvas: { scale: 2 }, jsPDF: { unit: 'in', format: 'letter' } };

  const pdfBlob = await html2pdf().set(opt).from(element).output('blob');

  const response = await fetch('/api/artifacts/generate-presigned-url', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ filename: `${state.title.replace(/\s+/g, '_')}.pdf`, contentType:
'application/pdf' })
  });
  const { uploadUrl, s3FileKey } = await response.json();

  const uploadResult = await fetch(uploadUrl, {
    method: 'PUT',
    headers: { 'Content-Type': 'application/pdf' },
    body: pdfBlob
  });

  if (uploadResult.ok) {
    copilotContext.sendActionExecutionNotification({
      name: "notifyArtifactSaved",
      arguments: { s3Url: `s3://enterprise-artifacts-bucket/${s3FileKey}`, status: "SUCCESS" }
    });
  } else {
    throw new Error("S3 Upload Transaction Failed");
  }
}
```

5. Advanced Resilience Guide (Zero-Failure Execution Framework)

Building automated loops around layout compilation requires strict guards against common agent failure modes: schema corruption, canvas rendering drops, and memory leaks.

Mandatory Design Principle: Self-Correction Loop Pattern

Never allow an agent to crash the user interface upon generating invalid structural syntax. Implement an intercept layer: if the JSON schema validation breaks or missing metrics occur, catch the exception, extract the exact validation failure stack trace, and automatically feed it back into the graph's self-correcting routing loop for execution retry.

5.1 Matrix of Core Failure Modes & Remediation Strategies

Identified Risk Vector	Root Systemic Cause	Technical Architecture Remediation Strategy
Schema Drift & Halting	Agent structural deviations; unexpected structural changes across LLM model upgrades.	Enforce strict runtime data validation on the client side using Zod or JSONSchema. If validation fails, intercept the process, isolate the specific missing or mismatched node, and feed a detailed error trace back to the self-correction routing loop.
Chart Canvas Rendering Clipping	Canvas components rendering asynchronously inside hidden tabs or animating during screenshot capture.	Disable initial entry animations on Recharts or Chart.js profiles inside export-designated contexts. Use explicit, explicit layout heights on wrapper nodes and execute DOM-to-PDF hooks only when the wrapper's <code>onAnimationEnd</code> or load lifecycle resolves.
AWS S3 Connection Drops	Network degradation, intermittent rate-limiting on high-density data batch runs.	Implement a standardized exponential backoff algorithm with jitter on all client-to-S3 operations. Wrap standard PUT calls in a 3-tier retry execution stack before failing upwards to user alert frames.
Memory Leaks on Long Runs	Accumulated DOM references and heavy base64 data URIs remaining pinned in client context.	Enforce strict manual garbage collection. Revoke structural blob memory pathways immediately using <code>URL.revokeObjectURL()</code> once S3 pushes settle or print processes successfully conclude.

5.2 Backend Gateway Pre-signed URL Strategy (Python Implementation)

To ensure strict access isolation, the application backend should grant short-lived write authorization using the official AWS SDK for Python (`boto3`).

```
import boto3
from botocore.exceptions import ClientError

def generate_secure_upload_url(bucket_name, object_name, expiration=900):
    s3_client = boto3.client('s3', region_name='ap-south-1')
    try:
        response = s3_client.generate_presigned_url(
            'put_object',
            Params={'Bucket': bucket_name, 'Key': object_name, 'ContentType': 'application/
pdf'},
            ExpiresIn=expiration
        )
    except ClientError as error:
        return None
    return response
```